

Lab 8: Table Relationships — Product Shop

วิชา: CP353002 Principles of Software Design
ผู้จัดทำ: นางสาวปภาวรินทร์ นาเมืองรักษ์ รหัสนักศึกษา 673380275-5 Section 2
หัวข้อ: ความสัมพันธ์ตาราง 1:1 และ 1:N ด้วย Spring Boot + JPA + PostgreSQL

สารบัญ

- 1. ส่วนที่ 1: หลักการออกแบบซอฟต์แวร์ (Software Design Principles)
 - ทบทวน SOLID Principles กับ Entity และ Relationships
 - ความแตกต่างระหว่าง 1:1 และ 1:N
 - Strategy Pattern ในการคำนวณส่วนลด
 - Execution Flow (HTTP Request → Controller → Service → Repository → DB)
- 2. ส่วนที่ 2: Code Implementation & คำอธิบายสถาปัตยกรรม
 - Layered Architecture Overview
 - Entity Models & JPA Annotations (Product, ProductDetail, Review)
 - Repositories (DIP & ISP)
 - Service Layer & Business Logic
 - Controller Layer & Constructor Injection
- 3. ส่วนที่ 3: ขั้นตอนการทดสอบและการเตรียมภาพหน้าจอสำหรับรายงาน (PDF Report)

ส่วนที่ 1: หลักการออกแบบซอฟต์แวร์ (Software Design Principles)

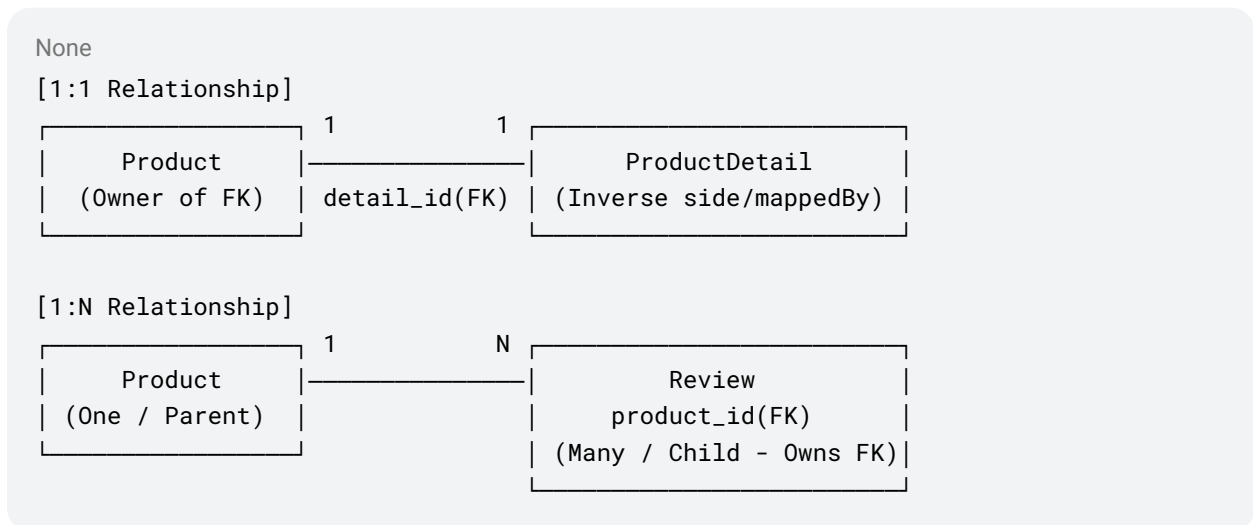
SOLID Principles กับการออกแบบ Entity & Relationship

หลักการ (Principle)	ความหมายทางทฤษฎี	การประยุกต์ใช้กับ Entity และ Relationship ในโปรเจกต์นี้
S — Single Responsibility Principle (SRP)	แต่ละคลาส/โมดูลควรมีหน้าที่และความรับผิดชอบเพียงอย่างเดียว	แยก ProductDetail ออกจาก Product โดยไม่นำฟิลด์ข้อมูลเสริม (เช่น warranty, weight, dimensions, manufacturedCountry) มารวมไว้ใน Product เพื่อให้

หลักการ (Principle)	ความหมายทางทฤษฎี	การประยุกต์ใช้กับ Entity และ Relationship ในโปรเจกต์นี้
		Product รับผิดชอบเฉพาะข้อมูลหลักของสินค้า และ ProductDetail รับผิดชอบข้อมูลเชิงเทคนิค
O — Open/Closed Principle (OCP)	เปิดรับการขยายฟังก์ชันใหม่ แต่ปิดรับการแก้ไขโค้ดเดิม	การออกแบบความสัมพันธ์แบบ 1:N กับ Review ทำให้ระบบสามารถเพิ่มข้อมูลรีวิวใหม่ ๆ ได้อย่างอิสระโดยไม่ต้องแก้ไข schema หรือ attributes ของ Product รวมถึงใน Strategy Pattern เมื่อต้องการเพิ่มส่วนลดใหม่ (เช่น VIP Discount) ก็สามารถสร้างคลาสใหม่ได้ทันที
L — Liskov Substitution Principle (LSP)	ซับคลาสหรือคลาสที่ Implement ต้องสามารถแทนที่ Superclass/Interface ได้อย่างสมบูรณ์โดยไม่ทำให้การทำงานผิดพลาด	คลาส Concrete Strategy ทุกตัว (NoDiscountStrategy, MemberDiscountStrategy, SeasonalSaleStrategy) สามารถนำไปแทนที่ผ่าน Interface DiscountStrategy ใน DiscountContext ได้ทันที โดยรับประกันผลลัพธ์เป็นตัวเลขราคาสุทธิที่ถูกต้อง
I — Interface Segregation Principle (ISP)	ไม่ควรบังคับให้ Client พึ่งพา Method ที่ตนเองไม่ได้ใช้งาน	แยก Repository Interface ออกจากกันเป็น ProductRepository, ProductDetailRepository, และ ReviewRepository แทนที่จะสร้างรวมเป็น Interface ขนาดใหญ่ และ Interface DiscountStrategy มีเฉพาะ method ที่จำเป็น (calculatePrice, getName)

หลักการ (Principle)	ความหมายทางทฤษฎี	การประยุกต์ใช้กับ Entity และ Relationship ในโปรเจกต์นี้
D — Dependency Inversion Principle (DIP)	โมดูลระดับสูงต้องไม่พึ่งพาโมดูลระดับต่ำโดยตรง แต่ทั้งคู่ต้องพึ่งพา Abstraction	ProductService พึ่งพา Repository Interfaces (ProductRepository, etc.) และ Strategy Interface โดย Spring ทำการฉีด (Constructor Injection) คลาส Implementation มาให้ตอนรันไทม์ ทำให้สามารถสลับ Database หรือเปลี่ยน Implementation ได้โดยไม่กระทบ Service

ความแตกต่าง 1:1 กับ 1:N



1. One-to-One (1:1) — Product ↔ ProductDetail

- **แนวคิด:** สินค้า 1 ชิ้น มีรายละเอียดเสริมได้เพียง 1 ชุดเท่านั้น
- **เหตุผลที่ใช้:** ข้อมูลเสริม (Description, Warranty, Dimensions, Weight, Country) ไม่ได้ถูกเรียกใช้งานในทุก Query (เช่น หน้าค้นหา หรือแสดงรายการสินค้าแบบย่อ) การแยกออกเป็นตารางต่างหากช่วยลดขนาด Row ใน Database, เพิ่มประสิทธิภาพ I/O และเป็นไปตามหลัก **SRP**
- **กฎ Foreign Key:** อยู่ที่ฝั่ง Owner คือตาราง products มีคอลัมน์ detail_id ชี้ไปยัง id ของ product_details

- ฝั่ง Product: ใช้ @OneToOne(cascade = CascadeType.ALL) + @JoinColumn(name = "detail_id")
- ฝั่ง ProductDetail: ใช้ @OneToOne(mappedBy = "detail") (Inverse side)

2. One-to-Many (1:N) — Product → Review

- **แนวคิด:** สินค้า 1 ชิ้น สามารถมีผู้ใช้งานเข้ามารีวิวและให้คะแนนได้ไม่จำกัดจำนวน (0 ถึง N รีวิว)
- **เหตุผลที่ใช้:** ข้อมูล Child ขยายตัวได้เรื่อย ๆ ตามกาลเวลา การออกแบบเช่นนี้ทำให้ระบบรองรับการเติบโตของข้อมูลตามหลัก **OCP** โดยไม่ต้องปรับโครงสร้างตารางหลัก
- **กฎ Foreign Key:** อยู่ฝั่ง Many เสมอ คือตาราง reviews มีคอลัมน์ product_id ชี้ไปยัง id ของ products
 - ฝั่ง Review (ฝั่ง Many เป็นเจ้าของ FK): ใช้ @ManyToOne + @JoinColumn(name = "product_id")
 - ฝั่ง Product: ใช้ @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)

Strategy Pattern ในการคำนวณส่วนลด

การคำนวณส่วนลดของสินค้านำ Strategy Pattern มาประยุกต์ใช้เพื่อหลีกเลี่ยงการเขียน Hard-coded if-else หรือ switch-case ซ้ำซ้อนใน Service Layer:

1. **Strategy Interface (DiscountStrategy):**
 - กำหนดสัญญา (Contract) ร่วมกัน: Double calculatePrice(Double originalPrice) และ String getName()
2. **Concrete Strategies:**
 - NoDiscountStrategy: ราคาปกติ ลด 0%
 - MemberDiscountStrategy: ส่วนลดสมาชิก ลด 10%
 - SeasonalSaleStrategy: ส่วนลดตามเทศกาล ลด 20%
3. **Context (DiscountContext):**
 - จัดการเลือก Strategy ที่สอดคล้องกับค่า discountType ของสินค้า แล้วมอบหมาย (Delegate) การคำนวณราคาให้ Strategy นั้น ๆ

ข้อดีด้าน Software Design: สอดคล้องกับ **OCP** อย่างแท้จริง หากในอนาคตต้องการเพิ่มกลยุทธ์ส่วนลด Flash Sale หรือ Black Friday เราสามารถสร้าง Class ใหม่ที่ implements DiscountStrategy ได้ทันทีโดยไม่ต้องแก้ไขโค้ดการทำงานของ ProductService

Execution Flow (การไหลของข้อมูล)

กรณี: ผู้ใช้ทำการเพิ่มสินค้าใหม่พร้อมรายละเอียด (Create Product with 1:1 Detail)

None

```
[1. User Browser]
  | POST /products/save (Form Data: Name, Price, Detail...)
  ▼
[2. ProductController] (Presentation Layer)
  | รับ Object Product และ ProductDetail ผ่าน @ModelAttribute
  | เรียก productService.saveProduct(product)
  ▼
[3. ProductService] (Business Logic Layer)
  | ตั้งค่าความสัมพันธ์ 1:1: detail.setProduct(product)
  | เรียก productRepository.save(product)
  ▼
[4. ProductRepository] (Data Access Layer - Spring Data JPA)
  | Hibernate แปลง Object เป็น SQL Commands:
  | - INSERT INTO product_details (...) RETURNING id
  | - INSERT INTO products (... , detail_id) VALUES (...)
  ▼
[5. PostgreSQL Database]
  | บันทึกข้อมูลจริงลง 2 ตารางพร้อมเชื่อมโยง Foreign Key (detail_id)
  ▲
  | ส่งสถานะสำเร็จกลับขึ้นมา
  ▼
[6. Redirect & Render]
  | Controller สั่ง redirect:/products พร้อมส่ง Flash Message
  | Browser ส่ง GET /products
  | Service ดึงข้อมูลทั้งหมดผ่าน productRepository.findAll()
  | Service เรียก DiscountContext คำนวณ finalPrice ผ่าน Strategy Pattern
  | Thymeleaf Template (list.html) ทำการ Render ข้อมูลออกเป็น HTML สวยงาม
```

ส่วนที่ 2: Code Implementation & คำอธิบายสถาปัตยกรรม

สถาปัตยกรรม Layered Architecture

โปรเจกต์แบ่งออกเป็น 4 เลเยอร์ที่แยกหน้าที่กันอย่างเด็ดขาด:

1. **Presentation Layer (controller/ + templates/)**: จัดการ HTTP Requests, Model Binding และ UI Presentation ด้วย Thymeleaf
2. **Business Logic Layer (service/)**: ควบคุม Transaction, Business Rules และคำนวณส่วนลดผ่าน Strategy Pattern
3. **Data Access Layer (repository/)**: สื่อสารกับ PostgreSQL ผ่าน Spring Data JPA
4. **Domain Model (model/)**: กำหนด Entities, Columns และ Relationships (1:1, 1:N)

None

```
src/main/java/com/example/demo/
├─ DemoApplication.java
├─ model/
│   ├── Product.java           ← 1:1 กับ ProductDetail, 1:N กับ Review
│   ├── ProductDetail.java     ← Inverse 1:1 (mappedBy="detail")
│   └─ Review.java            ← ฝั่ง Many เก็บ FK (product_id)
├─ repository/
│   ├── ProductRepository.java
│   ├── ProductDetailRepository.java
│   └─ ReviewRepository.java
├─ strategy/
│   ├── DiscountStrategy.java
│   ├── NoDiscountStrategy.java
│   ├── MemberDiscountStrategy.java
│   ├── SeasonalSaleStrategy.java
│   └─ DiscountContext.java
├─ service/
│   └─ ProductService.java
└─ controller/
    └─ ProductController.java
```

1. Entity Models & JPA Annotations

Product.java (เจ้าของความสัมพันธ์ 1:1 และ 1:N)

Java

```
@Entity
@Table(name = "products")
public class Product {
```

```
    @Id
```

```

@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

@Column(nullable = false)
private String name;

@Column(nullable = false)
private String category;

@Column(nullable = false)
private String brand;

@Column(nullable = false)
private Integer stock;

@Column(nullable = false)
private Double price;

@Column(name = "discount_type", nullable = false)
private String discountType = "NONE";

// — 1:1 กับ ProductDetail (Cascade ALL เพื่อให้บันทึก/ลบพร้อมกัน) —
@OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
@JoinColumn(name = "detail_id", referencedColumnName = "id")
private ProductDetail detail = new ProductDetail();

// — 1:N กับ Review (ฝั่ง One กำหนด mappedBy ซึ่งไปฟิลด์ product ใน Review) —
@OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval =
true)
private List<Review> reviews = new ArrayList<>();

// — Fields สำหรับ Strategy Pattern (ไม่บันทึกลง Database) —
@Transient
private String discountName;

@Transient
private Double finalPrice;

// ... getters, setters & helper methods ...
}

```

ProductDetail.java (Inverse side ของ 1:1)

```
Java
@Entity
@Table(name = "product_details")
public class ProductDetail {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 1000)
    private String description;

    @Column(length = 100)
    private String warranty;

    private Double weight;

    @Column(length = 100)
    private String dimensions;

    @Column(name = "manufactured_country", length = 100)
    private String manufacturedCountry;

    // Inverse side ของความสัมพันธ์ 1:1 อ้างอิง property 'detail' ใน Product
    @OneToOne(mappedBy = "detail")
    private Product product;

    // ... getters and setters ...
}
```

Review.java (ฝั่ง Many ของ 1:N เก็บ FK)

```
Java
@Entity
@Table(name = "reviews")
public class Review {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
```



```
private String reviewer;

@Column(nullable = false)
private Integer rating;

@Column(length = 1000)
private String comment;

@Column(name = "review_date", nullable = false)
private LocalDate reviewDate;

// Foreign Key 'product_id' จัดเก็บในตาราง reviews
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "product_id", nullable = false)
private Product product;

// ... getters and setters ...
}
```

2. Repository (Data Access Layer)

นำ Spring Data JPA มาใช้ผ่าน Interface ที่ขยาย JpaRepository ทำให้ได้ฟังก์ชัน CRUD ชั้นพื้นฐานโดยไม่ต้องเขียน SQL เอง (ISP & DIP):

- ProductRepository extends JpaRepository<Product, Long>
- ProductDetailRepository extends JpaRepository<ProductDetail, Long>
- ReviewRepository extends JpaRepository<Review, Long>

3. Service Layer (Business Logic)

คลาส ProductService ทำหน้าที่เป็นผู้ประสานงานหลักระหว่าง Repositories และ Strategy Pattern:

- **Constructor Injection:** รับ ProductRepository, ProductDetailRepository, ReviewRepository, และ DiscountContext ผ่าน Constructor ช่วยให้การทดสอบ (Unit Test) ทำได้ง่าย

- **CRUD Operations:** รองรับการสร้าง อัปเดต แสดงรายการ และลบสินค้า โดยอาศัย `CascadeType.ALL` ทำให้ข้อมูลใน `ProductDetail` อัปเดตหรือถูกลบตาม `Product` ได้โดยอัตโนมัติ

4. Controller (Presentation Layer)

คลาส `ProductController` รองรับ URL Mappings ตามที่ระบุไว้ในคู่มือ Lab 8:

HTTP Method	URL Path	คำอธิบาย
GET	/products	แสดงรายการสินค้าทั้งหมดพร้อมราคาส่วนลดและเรตติ้ง (list.html)
GET	/products/add	แสดงฟอร์มเพิ่มสินค้าใหม่และรายละเอียดเสริม (add.html)
POST	/products/save	บันทึกข้อมูลสินค้าใหม่และรายละเอียด <code>ProductDetail</code>
GET	/products/edit/{id}	แสดงหน้าฟอร์มแก้ไขข้อมูลสินค้าเดิม (edit.html)
POST	/products/update/{id}	บันทึกการอัปเดตข้อมูลสินค้า
GET	/products/delete/{id}	แสดงหน้าจอยืนยันการลบสินค้า (delete.html)
POST	/products/delete/{id}	ลบสินค้าออกจากฐานข้อมูล พร้อม Cascade ลบ Detail และ Review

ส่วนที่ 3: ขั้นตอนการรันและเตรียมภาพหน้าจอ

การตั้งค่า Database ในเครื่อง

1. เข้า PostgreSQL และสร้างฐานข้อมูล lab8shop:

Shell

```
psql -U postgres
CREATE DATABASE lab8shop;
\q
```

2. ตรวจสอบรหัสผ่านใน src/main/resources/application.properties ให้ตรงกับ PostgreSQL ในเครื่อง (ค่าปัจจุบันตั้งไว้ 1234)
3. สั่งรันโปรเจกต์:

Shell

```
mvn spring-boot:run
```

(Hibernate จะสร้างตาราง *products*, *product_details*, *reviews* พร้อม *Foreign Key* ให้อัตโนมัติ)

4. เข้าใช้งานแอปพลิเคชันผ่านเว็บเบราว์เซอร์ที่: <http://localhost:8080/products>

รายการภาพหน้าจอที่ต้องเตรียมสำหรับทำ PDF รายงาน

💡 **หมายเหตุสำคัญ:** เพื่อความถูกต้องตามเกณฑ์การประเมิน ให้ระบุชื่อสินค้าเป็นรูปแบบ: iPhone 15 Pro (673380275-5 SEC 2)

1. **Create Screen:** ถ่ายภาพหน้า /products/add ขณะกรอกข้อมูลสินค้าและข้อมูลเสริม (ProductDetail) ครบถ้วน

← กลับไปรายการสินค้า

⊕ เพิ่มสินค้าใหม่

กรอกข้อมูลสินค้าหลักและรายละเอียดเชิงลึก (Product & 1:1 ProductDetail)

1. ข้อมูลสินค้าหลัก (Product)

ชื่อสินค้า (ใส่รหัสสินค้าและ Section สำหรับรายงาน)
Fujitsu arrows tab q738(673380275-5 sec2)

หมวดหมู่ (Category)
Tablet

แบรนด์ (Brand)
Fujitsu

ราคาปกติ (บาท)
28000

จำนวนในคลัง (Stock)
2

% ส่วนลด (Strategy Pattern)
ราคาปกติ (0%)

2. ข้อมูลเสริมสินค้า (1:1 ProductDetail)

รายละเอียดสินค้า (Description)
Windows Tablet

การรับประกัน (Warranty)
3 mounth

น้ำหนัก (Weight in kg)
1.3 kg

ขนาด (Dimensions)
หน้าจอขนาด 13 นิ้ว

ประเทศที่ผลิต (Manufactured Country)
China

บันทึกสินค้า ยกเลิก

2. **Read Screen:** ถ่ายภาพหน้ารายการสินค้า /products ที่แสดงสินค้าพร้อมคำนวณราคาสุทธิ (Strategy Pattern), ข้อมูลการรับประกัน (1:1), และรีวิว (1:N)

http://localhost:8080/products

VPN

Product Shop

CP353002 — Lab 8: Spring Boot + JPA + PostgreSQL (1:1 & 1:N)

รายการสินค้าทั้งหมด

จัดการข้อมูลสินค้าในระบบ — รองรับการสัมพันธ์ 1:1 (Detail) และ 1:N (Reviews)

เพิ่มสินค้าใหม่

เพิ่มสินค้า "Fujitsu arrows tab q738(673380275-5 sec2)" สำเร็จ

#	ชื่อสินค้า	หมวดหมู่	แบรนด์	คลัง	โปรโมชั่น (STRATEGY)	ราคาสุทธิ (บาท)	รายละเอียดเสริม (1:1)	รีวิว (1:N)	จัดการ
1	Sony Xperia I Mark3	Electronics	Sony	5 ชิ้น	ส่วนลดสมาชิก (10%)	40500.00 ฿ 45000.00 ฿	✓ 1 Year 📍 Japan 📦 0.2 kg	★ 0.0 (0)	 
2	Fujitsu arrows tab q738(673380275-5 sec2)	Tablet	Fujitsu	2 ชิ้น	ราคาปกติ (0%)	28000.00 ฿	✓ 3 month 📍 China 📦 1.3 kg	★ 0.0 (0)	 

ทั้งหมด 2 รายการ

CP353002 Principles of Software Design — Lab 8: Table Relationships (1:1 & 1:N)

3. Update Screen: ถ่ายภาพหน้า /products/edit/{id} ขณะแก้ไขข้อมูลสินค้า

http://localhost:8080/products/edit/9

VPN

Product Shop

CP353002 — Lab 8

← กลับไปรายการสินค้า

แก้ไขข้อมูลสินค้า

แก้ไขข้อมูลสินค้าและรายละเอียดเสริม: Fujitsu arrows tab q738(673380275-5 sec2)

1. ข้อมูลสินค้าหลัก (Product)

ชื่อสินค้า

Fujitsu arrows tab q738(673380275-5 sec2)

หมวดหมู่ (Category)

Tablet

แบรนด์ (Brand)

Fujitsu

ราคาปกติ (บาท)

28000.0

จำนวนในคลัง (Stock)

10

% ส่วนลด (Strategy Pattern)

ส่วนลดเทศกาล (ลด 20%)

2. ข้อมูลเสริมสินค้า (1:1 ProductDetail)

รายละเอียดสินค้า (Description)

Windows Tablet

การรับประกัน (Warranty)

3 month

น้ำหนัก (Weight in kg)

1.3

ขนาด (Dimensions)

หน้าจอขนาด 13 นิ้ว

ประเทศที่ผลิต (Manufactured Country)

China

อัปเดตข้อมูล

ยกเลิก

4. Delete Screen: ถ่ายภาพหน้ายืนยันการลบ /products/delete/{id} และหน้ารายการสินค้าหลังลบสำเร็จ

http://localhost:8080/products/delete/8

Product Shop

CP353002 — Lab 8

← กลับไปรายการสินค้า

ยืนยันการลบสินค้า



คุณแน่ใจหรือไม่ที่จะลบสินค้านี้?

การดำเนินการนี้จะลบข้อมูลสินค้าพร้อมรายละเอียด (ProductDetail) และรีวิวทั้งหมด (Cascade Delete)

ชื่อสินค้า	Sony Xperia I Mark3
หมวดหมู่ / แบนด์	Electronics Sony
ราคาปกติ	45000.00 บาท
ราคาสุทธิ (Strategy)	40500.00 บาท
การรับประกัน (1:1)	1 Year
ประเทศผลิต (1:1)	Japan
จำนวนรีวิว (1:N)	0 รายการ

 ยืนยันลบสินค้า  ยกเลิก

```
lab8shop=# SELECT * FROM products;
 id | brand | category | discount_type | name | price | stock | detail_id
-----+-----+-----+-----+-----+-----+-----+-----
  9 | Fujitsu | Tablet | SEASONAL | Fujitsu arrows tab q738(673380275-5 sec2) | 28000 | 10 | 9
(1 row)
```

5. **Database Structure (pgAdmin / DBeaver / psql):** ถ่ายภาพ Diagram หรือตารางทั้ง 3 (products, product_details, reviews) ที่แสดงคอลัมน์ Foreign Key (detail_id และ product_id) อย่างชัดเจน

lab8shop-# \d product_details

Table "public.product_details"				
Column	Type	Collation	Nullable	Default
id	bigint		not null	generated by default as identity
description	character varying(1000)			
dimensions	character varying(100)			
manufactured_country	character varying(100)			
warranty	character varying(100)			
weight	double precision			

Indexes:

"product_details_pkey" PRIMARY KEY, btree (id)

Referenced by:

TABLE "products" CONSTRAINT "fka9j512fdisl3ol4ix883w8bcj" FOREIGN KEY (detail_id) REFERENCES product_details (id)

lab8shop-# \d products

Table "public.products"				
Column	Type	Collation	Nullable	Default
id	bigint		not null	generated by default as identity
brand	character varying(255)		not null	
category	character varying(255)		not null	
discount_type	character varying(255)		not null	
name	character varying(255)		not null	
price	double precision		not null	
stock	integer		not null	
detail_id	bigint			

Indexes:

"products_pkey" PRIMARY KEY, btree (id)

"ukfq6xkbp2dny2wua6wcu5jcwj" UNIQUE CONSTRAINT, btree (detail_id)

Foreign-key constraints:

"fka9j512fdisl3ol4ix883w8bcj" FOREIGN KEY (detail_id) REFERENCES product_details(id)

Referenced by:

TABLE "reviews" CONSTRAINT "fkpl51cejpw4gy5swfar8br9ngi" FOREIGN KEY (product_id) REFERENCES products(id)

lab8shop-# \d reviews

Table "public.reviews"				
Column	Type	Collation	Nullable	Default
id	bigint		not null	generated by default as identity
comment	character varying(1000)			
rating	integer		not null	
review_date	date		not null	
reviewer	character varying(255)		not null	
product_id	bigint		not null	

Indexes:

"reviews_pkey" PRIMARY KEY, btree (id)

Foreign-key constraints:

"fkpl51cejpw4gy5swfar8br9ngi" FOREIGN KEY (product_id) REFERENCES products(id)

lab8shop-#